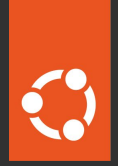
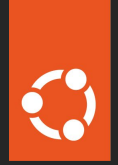


LPC 2025: seccomp listeners for nested containers

Aleksandr Mikhalitsyn
12.12.2025

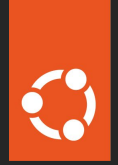


Intro



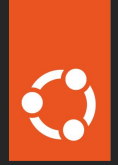
Intro (basics)

- SECCOMP_SET_MODE_STRICT
  - only read/write/exit/sigreturn/u{,ret}probe are allowed
- SECCOMP_SET_MODE_FILTER (cBPF program)
    - receives (struct seccomp_data) as a data (nr, arch, args, IP)
    - makes a decision (allow, log, trace, user_notif, errno, trap, kill)



Intro (filter mode)

1. seccomp filters are represented by (struct seccomp_filter)
2. inherited during fork()
3. organized in a tree via unidirectional linked list (->prev field)
4. current->seccomp.filter is (struct seccomp_filter *)
5. (struct seccomp_filter) is stable
6. (struct seccomp_filter*)->prog is a classical BPF program loaded



Intro (filter mode) cont.

In SECCOMP_SET_MODE_FILTER mode we:

1. start from current->seccomp.filter and walk up the tree (via ->prev)
2. execute cBPF program for every single filter (unless cache is available)
3. find a **strictest** result from (allow, log, trace, user_notif, errno, trap, kill)
4. enforce it

Intro (user notify)

seccomp filter mode with SECCOMP_FILTER_FLAG_NEW_LISTENER flag:

- returns a notification/listener file descriptor
- when result is SECCOMP_RET_USER_NOTIF kernel sends a notification to the userspace
- userspace daemon can do ioctl(SECCOMP_IOCTL_NOTIF_RECV / SECCOMP_IOCTL_NOTIF_SEND)
  - At this time between RECV/SEND userspace may implement a syscall in userspace. And even install file descriptors into fdtable (see SECCOMP_IOCTL_NOTIF_ADDFD)!
  - … or just say SECCOMP_USER_NOTIF_FLAG_CONTINUE

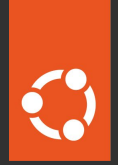
Intro (when we need listeners)

- Container runtimes like LXC are doing syscall interception to allow more
  - mknod() syscall in user namespace (requires CAP_MKNOD in initial user namespace)
  - “virtualize” sysinfo() syscall
- “fooling” of an application inside the container by emulating desired behavior (for example in Anbox = Android in a Box)

Intro (problem of nesting)

Can’t have more than one seccomp filter with listener fd.

- container runtime may want to setup interception
- user application (or nested container runtime) inside may want too
  - => can’t do that (EBUSY from seccomp())



Implementation

Patchset in LKML (v3)



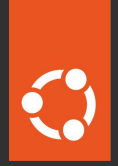
Naive approach

1. nested listener limitation seems redundant on the first look
   a. seccomp filters are **naturally** nested
   b. only listeners are not allowed to
2. Can we just remove the limitation and that's it?.. Of course, not :)
3. userspace may reply with SECCOMP_USER_NOTIF_FLAG_CONTINUE
   a. What if we have another listener up the tree?
      i. => We **must** consult with it! Otherwise, it's a clear seccomp-bypass scenario.

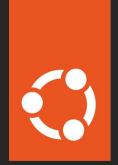
The proper way

Assume we've run filters normally, and the final result is "user notify".

1. Send notification to the last filter in the tree that returned "user notify"
    a. Wait for a reply
        i. if reply is "return" -> just return
        ii. if reply is "continue" -> *go up the tree and ask an upper-level listener of the filter returned "user notify"*



Pitfalls



Problem 1 (“many user_notify”)

```
struct sock_filter filter[] = {
    BPF_STMT(BPF_RET|BPF_K, SECCOMP_RET_USER_NOTIF),
};

struct sock_fprog p = {
    .len = (unsigned short)ARRAY_SIZE(filter),
    .filter = filter,
};
```

Problem 1 (“many user_notify”) cont.

1. fd = seccomp(SECCOMP_<..>_FILTER, SECCOMP_<..>_LISTENER, &p);
2. close(fd);

1. seccomp(SECCOMP_<..>_FILTER, 0, &p)

… and now kernel state is the same! Why?

- (struct seccomp_filter *)->notif is a pointer to (struct notification)
  - allocated during seccomp() **only** if userspace asks for a listener
  - released from (struct file_operations *)->release() callback and NULLified!
- cBPF program can return SECCOMP_RET_USER_NOTIF (no matter if listener is there)
  - ENOSYS from a syscall if notification delivery has failed

Problem 1 (“many user_notify”) cont.

1. fd1 = seccomp(<F>, <**L**>, &p);
2. fd2 = seccomp(<F>, <**L**>, &p);
3. fail with EBUSY

1. fd1 = seccomp(<F>, <**L**>, &p);
2. close(fd1);
3. fd2 = seccomp(<F>, <**L**>, &p);
4. OK

1. seccomp(<F>, **0**, &p);
2. fd = seccomp(<F>, <**L**>, &p);
3. OK

Problem 1 (“many user_notify”) cont.

1. seccomp(<F>, **0**, &p);

2. No notify fd, fail on attempt to send a notification -> ENOSYS
3. *wake up our victim process*

Process under seccomp

1. calls getpid()

2. *sleeps*

3. wakes up and see ENOSYS

Problem 1 (“many user_notify”) cont.

1. seccomp(<F>, **0**, &p);
2. fd = seccomp(<F>, <L>, &p);

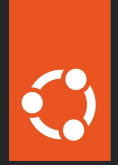
3. ioctl(fd, <RECV>, ..)
4. *syscall continue*
5. ioctl(fd, <SEND>, ..)
6. *wake up our victim process*

seccomp(<F>, 0, &p) has no effect here!

Process under seccomp

1. calls getpid()
2. *sleeps*

3. native “in-kernel” getpid() implementation is called

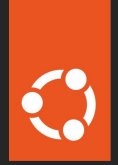


Problem 1 (“many user_notify”) cont.

Described behavior is not intuitive at all (at least, for me :)), but we have to (or not?) ensure that we keep it otherwise it is a (small) uAPI breakage.

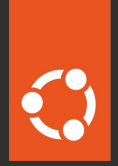
To deal with this I had to:

- start distinguishing between closed-listener and no-listener cases (convert ->notif pointer accesses to use IS_ERR_OR_NULL() and friends)
- start tracking who was the first listener installed in the tree to provide an old behavior (ignoring all filters without listeners up the tree)



Problem 2 (performance)

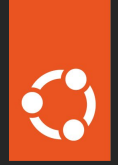
- seccomp logic needs to be amazingly fast, because __secure_computing() is called on every syscall entry when seccomp is enabled for a task
  - search for SYSCALL_WORK_SECCOMP
- => we need to avoid modifying a “fast-path” when no listeners involved (thanks to Andrei Vagin for a private discussion)



Current design

Current design

- Introduce SECCOMP_FILTER_FLAG_ALLOW_NESTED_LISTENERS flag to explicitly allow nested listeners
- No modifications on the fastpath
- Keep old quirky behavior everywhere possible



Thanks to

- Tycho Andersen
- Kees Cook
- Andrei Vagin



Thank you! Questions?